
Assessing Data Quality, Part II

Quantifiable measurements of specific data quality variables must be performed. These will be used in the diagnostic approach in which the subjective survey evaluations are compared to the quantitative metrics. In this chapter we focus on these quantitative metrics. When developing these measures, the company must determine what is to be measured: that set of data quality dimensions that are important to its mission and operations. Each organization must examine its operations and make its own determination.

We use the traditional database integrity constraints proposed by Codd (1970) and a subset of data quality dimensions (Wand and Wang 1996; Wang and Strong 1996) as the basis for our discussion of data quality metrics. We hope that the metrics discussed here will be usable of and in themselves and also serve as a basis on which the reader can develop new metrics to fit the organization's needs. Many of the formulations presented here are symbolic representations of metrics suggested by Pipino, Lee, and Wang (2002).

Codd Integrity Constraints

The Codd integrity constraints consist of entity integrity, referential integrity, domain integrity, and column integrity. Codd introduced a fifth, all-purpose category that he labeled business rules. This category is intended to capture integrity rules specific to an organization. The metrics developed to measure adherence to Codd's first four integrity constraints can be viewed as task-independent metrics. They reflect the states of the data without the contextual knowledge of the application.

They can be applied to any set of data regardless of the task or application at hand.

1

Entity integrity requires that no primary key field value in a table be null. A metric that reflects the degree of adherence to this rule can be the following:

Degree of adherence to entity integrity = $1 - (\text{Number of null primary keys} / \text{Total number of rows})$

2

The referential integrity rule states that the value of a foreign key in a table must match a value of a primary key in a designated related table, or the value of the foreign key must be null. Similar to the previous metric, a measure of adherence to the referential integrity rule would be

Degree of adherence to reference integrity = $1 - (\text{Number of nonmatching values excluding nulls in the dependent table} / \text{Total rows in the dependent table})$

3

Metrics for column integrity can be represented in a similar fashion. Column integrity requires that the values in the column be drawn from the set of permissible values. Again, a simple ratio reflecting the percentage of rows that adhere to this rule can be used:

Degree of adherence to column integrity = $1 - (\text{Number of invalid column values} / \text{Number of rows in table})$

Note that the preceding metrics are simple ratios and adhere to the following general form:

$$\text{Rating} = 1 - \left(\frac{\text{Number of undesirable outcomes}}{\text{Total outcomes}} \right)$$

The form follows the convention that 1 represents the most desirable and 0 the least desirable score. Typically, individuals are inclined to count exceptions to the rule rather than conformance with the rule. In addition, one would expect few exceptions, thus making the tabulation easier. The metric, however, should reflect the degree of adherence. Hence, for most metrics that are simple ratios, we suggest using the canonical form. In what follows, we extend our discussion to other useful dimensions and suggested metrics for their assessment.

Canonical form depends on context

Metrics for Data Quality

Basic Forms

Assessment of data quality requires assessments along a number of dimensions. Each organization must determine which of these dimensions are important to its operations and precisely define the variables that constitute the dimensions. These are the variables that will be measured. Many of the dimensions are multivariate in nature. Which variables are important to the firm must be clearly identified and defined.

Choosing the specific variables to measure can be much more difficult than defining the specific metric, which often reduces to the ratio form. In addition, many of these variables are context-dependent. Although falling within a specific dimensional category, the specific measure to assess a specific dimension will vary from organization to organization.

We examine a number of dimensions that can be used to assess data quality. We have chosen to discuss a set of dimensions that have been shown to be of particular interest and importance to many organizations.

Free of Error It is a common practice to use the term *accuracy* when referring to whether the data is correct. The dimension of accuracy itself, however, can consist of one or more variables, only one of which is whether the data are correct. We use *free of error* to label the dimension that represents whether the data is correct. If one is counting the number of data units in error, then the metric is

$$\text{Free-of-error rating} = 1 - \left(\frac{\text{Number of data units in error}}{\text{Total number of data units}} \right)$$

This formulation requires additional refinements. It certainly will require additional specificity based on context. A concise description of what constitutes a data unit is required. Is it a field, a record, a table? Further, what exactly constitutes an error? The sense of the precision of the measurement must be specified. Quite possibly, in one case an incorrect character in a text string is tolerable, whereas in another circumstance all characters must be correct. These are distinctions that make the metric context-dependent and that demand unambiguous definition. We cannot

stress enough the importance of developing definitions that span databases and stakeholders.

Completeness Another useful dimension is the completeness of the data. The completeness dimension can be viewed from at least three perspectives: schema completeness, column completeness, and population completeness. By schema completeness, we mean the degree to which entities and attributes are not missing from the schema. By column completeness, we mean the degree to which there exist missing values in a column of a table. Codd's column integrity, described earlier, can be considered a check of column completeness in the instance when a column represents a required property of an object. By population completeness, we mean the degree to which members of the population that should be present are not present. For example, if a column should contain at least one occurrence of all 50 states, but the column contains only 43 states, then the population is incomplete.

Each of the completeness types can be measured by the following simple ratio:

$$\text{Completeness rating} = 1 - \left(\frac{\text{Number of incomplete items}}{\text{Total number of items}} \right)$$

Consistency The dimension of consistency can also be viewed from a number of perspectives. For example, one can be concerned with the consistency of redundant data in one table or in multiple tables. Codd's referential integrity constraint is an instantiation of this type of consistency. A second type would be the consistency between two related data elements. For example, the name of the city and the postal code should be consistent. This can be enabled by entering just the postal code and filling in the name of the city systematically through the use of referential integrity with a postal code table. A third form of consistency focuses on consistency of format for the same data element used in different tables. This may or may not be a requirement and will depend on context. The metric to measure any of these variables could take the following form:

Consistency rating

$$= 1 - \left(\frac{\text{Number of instances violating specific consistency type}}{\text{Total number of consistency checks performed}} \right)$$

Aggregate Forms

In the previous section, different types of completeness and consistency were discussed. It may be necessary to assess each category of the dimension and then generate an aggregate. In this case the values of n single variable metrics must be aggregated.

To handle dimensions that require the aggregation of multiple data quality indicators (variables), the minimum or maximum operation can be applied. The argument would consist of the individual data quality indicators for the multivariate dimension. The min operator is conservative in that it assigns to the dimension an aggregate value no higher than the value of its weakest data quality indicator (evaluated and normalized to between 0 and 1). The individual variables may be measured using a simple ratio. The maximum operation would be used if a liberal interpretation is warranted.

Let M_i represent the normalized value of the i th variable. Then the metric, using the min operator, takes the form $\min(M_1, M_2, \dots, M_n)$, where $M_1, M_2, \dots, M_n$ are normalized assessments of the variables of the dimension. The max operation takes the form $\max(M_1, M_2, \dots, M_n)$, where $M_1, M_2, \dots, M_n$ are normalized assessments of the variables of the dimension.

An alternative to the min operator is a weighted average of variables. For example, if a company has a good understanding of the importance of each variable to the overall evaluation of a dimension, then a weighted average of the variables would be appropriate. Formally, we would have

$$Rating = \sum_{i=1}^n a_i M_i$$

where a_i is a weighting factor, $0 \leq a_i \leq 1$, and $a_1 + a_2 + \dots + a_n = 1$. M_i is a normalized value of the assessments of the i th variable.

Two examples of dimensions that can make use of the min operator are believability and appropriate amount of data.

Believability Believability is the extent to which data is regarded as true and credible. As a rating of believability, one could use the subjective rating obtained as part of the IQA survey described in chapter 3. Alternatively, one might wish to define believability as a function of multiple variables. In this latter case either the min operation or the weighted

average might be used to arrive at an aggregate rating. For example, believability may reflect an individual's assessment of the credibility of the source of the data, the perceived timeliness of the data, or assessment against a common standard. One might define believability as follows:

Believability = $\min(\text{Believability of source}, \text{Believability when compared to internal commonsense standard}, \text{Believability based on age of data})$

Each of these variables would be rated on a scale from 0 to 1. The overall believability would then be assigned as the minimum value of the three. Assume that the believability of the source is rated 0.6, on a commonsense standard is rated 0.8, and on age is rated 0.7; then the overall believability rating would be $\min(0.6, 0.8, 0.7) = 0.6$.

One must beware of using a weighted average for a dimension such as believability. It is easy to assume that one is dealing with interval data and to subject the data to operations that are appropriate for interval or ratio scale data. If in fact one is dealing with ordinal data, the manipulations are not appropriate, and the numerical results could lead to misinterpretation.

Appropriate Amount of Data The min operator can also be used to calculate a measure for the appropriate amount of data. A working definition of the appropriate amount of data should reflect the state that the amount of data is neither too little nor too much. A metric would be

Rating of appropriate amount of data

$$= \min \left[\frac{\text{Number of data units provided}}{\text{Number of data units needed}}, \frac{\text{Number of data units needed}}{\text{Number of data units provided}} \right]$$

Complex Forms

Two dimensions of particular importance in practice are timeliness and accessibility. The metrics to measure these dimensions can be quite straightforward or can be modeled using a more complex formulation. We examine each briefly.

Timeliness The timeliness dimension reflects how up-to-date the data is with respect to the task for which it is used. A general metric to measure timeliness has been proposed by Ballou et al. (1998):

$$\text{Timeliness rating} = \left\{ \max \left[\left(1 - \frac{\text{Currency}}{\text{Volatility}} \right), 0 \right] \right\}^s$$

where currency = (delivery time – input time) + age, volatility refers to the length of time over which the data remains valid, delivery time refers to the time at which the data was delivered to the user, input time refers to the time at which the data was received by the system, and age refers to the age of the data when it was first received by the system. The exponent's value is task-dependent. It allows one to control the sensitivity of timeliness to the ratio. For example, notice the rating is 0.81 without a sensitivity adjustment ($s = 1$), 0.49 when $s = 2$ (more sensitive and thereby becomes less timely faster), and 0.9 when $s = 0.5$ (less sensitive).

For some organizations, such a complex measure is not necessary. For example, the age of the data may be an adequate measure of timeliness. Again these metrics are dependent on the context in which they will be applied.

Accessibility The accessibility dimension reflects the ease of attainability of the data. A metric that emphasizes the time aspect of accessibility can be defined as follows:

$$\text{Accessibility rating} = \left\{ \max \left[\left(1 - \frac{\text{Interval of time from request by user to delivery to user}}{\text{Interval of time from request to time at which no longer of any use}} \right), 0 \right] \right\}^s$$

If the data is delivered just prior to the time at which it is no longer of use, it may be of some use but not as useful as if it had been delivered much earlier. This metric trades off the time interval over which the user needs the data against the amount of time it takes to deliver the data. Here the time to obtain data increases until the first term goes negative, at which time accessibility is rated zero.

The metric uses time as the basic measure of accessibility. One can also define accessibility based on the structure and relations of the data paths and path lengths in other applications. As always, if time, structure, and path lengths all are considered important, then individual metrics for each can be developed, and an overall measure using the min operator can be defined.

The general form of the foregoing complex metrics makes use of the max operator. The general form is as follows:

$$Rating = \left\{ \max \left[\left(1 - \frac{Variable\ 1}{Variable\ 2} \right), 0 \right] \right\}^s$$

This form is useful when the dimension is a function of two variables, each of which participates in a trade-off. The exponent s is a sensitivity parameter.

Automating the Metrics

An outgrowth of MIT's Total Data Quality Management Research Program, the Integrity Analyzer (CRG 1997b) combines a TDQM methodology of define, measure, analyze, and improve with Codd's database integrity constraints of domain, entity, referential, column, and user-defined integrity.

We illustrate how the Integrity Analyzer could be used to obtain a rating of adherence to entity integrity. The entity integrity function checks that all primary and candidate keys for each table are unique and non-null.

To check entity integrity, the Integrity Analyzer user selects Entity Integrity from the data integrity pull-down menu and selects the Define tab. In the Define list box the user selects the fields that are primary and candidate keys for each table. Next, the user selects the Measure tab and asks the system to measure the number of violations of entity integrity. After the assessment has been completed, the user can select the Analyze tab and have the violation statistics displayed in numerical, graphical, or report form. Selecting the Improve tab produces a data object that displays the violation instances. The Referential integrity check works in a similar manner.

Screens corresponding to the functions of defining, measuring, analyzing, and improving entity integrity are shown in figure 4.1.

Similar procedures would be followed to perform referential and column integrity checks. To perform checks based on specific business rules, the user-defined integrity feature would be used.

Having chosen the user-defined integrity function, the user is presented with a display of conditions that have been defined. The user can now edit an existing condition or add a new user-defined rule. If the user

selects Add, the system displays a Define window that elicits the information necessary to define a condition.

As in the previous examples, selection of the Measurement function evaluates the database for violation of this rule. Selection of the Analysis function displays the results of the assessment. Selection of the Improvement function results in a display of the records that violate the condition.

The user-defined constraints, however, are application-dependent. They vary from application to application and from industry to industry. Furthermore, these application-dependent constraints evolve over time. The Integrity Analyzer can be customized for a specific company within a specific industry by coding these user-defined rules in the software.

The Integrity Analyzer is more than simply an implementation of Codd's integrity constraints in relational database software. Unlike the standard commercial relational database management systems (RDBMS) packages that check for adherence to Codd's constraints when data are entered into the database, the Integrity Analyzer is a diagnostic tool that can assess the degree to which existing databases adhere to all the constraints defined by Codd as well as application-dependent user-defined rules. As a diagnostic tool, it delivers an analysis of the data repository's current quality state and suggests where improvements can be made.

Clearly, software to perform the preceding functions can be developed in-house as part of the database function. The purpose of the example was to illustrate how some of the metrics can be assessed and analyzed in an automated fashion. How each organization chooses to implement the subjective and quantitative measurements remains a decision within the organization.

Here we have focused on the measures that can be used to assess specific variables or dimensions of data quality. The amount of data that must be assessed can be quite voluminous, however. In chapter 5 we present an overview of principles and techniques that can be applied to assessing a large database repository using sampling techniques.

Process-Embedded Data Integrity Approach

In chapter 3 we briefly introduced the process-embedded data integrity approach. Here we elaborate on this technique by illustrating how a

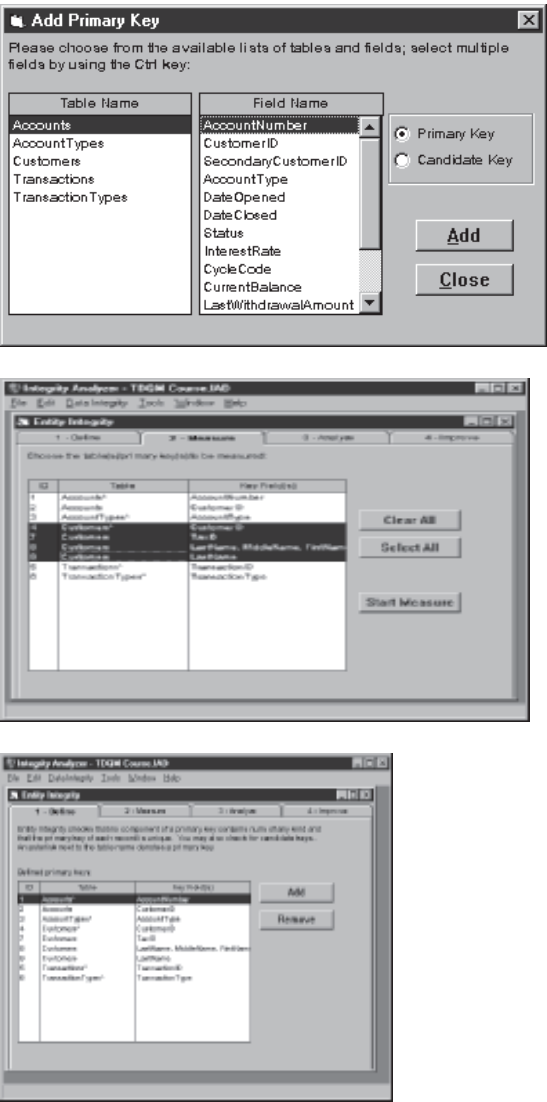
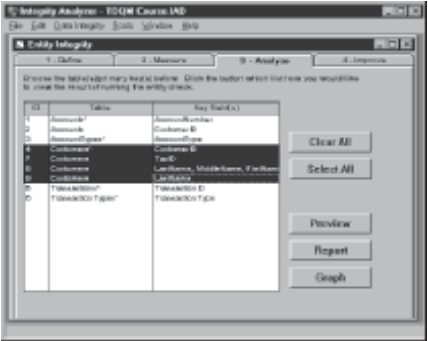
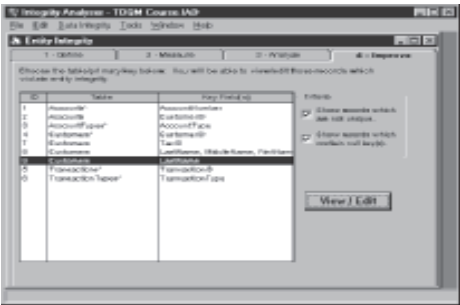


Figure 4.1
Entity Integrity Implementation
Source: Cambridge Research Group (CRG 1997b).



Entity Integrity - Violation Records						
CustomerID	Table	First Name	Middle Name	Last Name	City	State
763640854319913	AccountB	Laundres M		Barron	DELRAY BEACH	FL
2123691967099428	AccountB	Colores		Barron	WILMINGTON	MA
557536784322895	AccountOpen	Jeffrey		Beggs	CASSELBERRY	FL
7548234614011485	CustomerB	Jeff		Beggs	PLEASANT VALLEY	NY
081418860203382	CustomerB	Leandra		Boswell	WOODSTOCK	GA
60364368953326	CustomerB	Leandra		Boswell	WOODSTOCK	GA
810883087273862	CustomerB	Kylin		Brooks	TOWNS	VA
5705364972522889	CustomerB	K. David		Brooks	SHIRAZ GAN	NY
2763786437011895	CustomerB	Kylin		Brooks	BENSALEM	PA

Entity Integrity Results Preview						
	ID	Date	Time	TableSize	# Not Unique	# Null
▶	4	6/10/97	1:44:26 PM	1180	0	0
	7	6/10/97	1:44:26 PM	1180	0	0
	8	6/10/97	1:44:26 PM	1180	34	1180
	9	6/10/97	1:44:26 PM	1180	278	0
						278

Figure 4.1 (continued)

global manufacturer of consumer goods, Glocom, made use of it. The example is excerpted from the larger action research study that appeared in Lee et al. (2004).

Glocom, a consumer goods manufacturing company with subsidiaries in 52 countries, produces health, cleansing, and household products. Its corporate culture is friendly, international, and nonconfrontational. Glocom's information technology infrastructure is advanced; it typically adopts new information practices and technology early. It has up-to-date database systems and has been experimenting with data quality improvement projects for several years.

Glocom's first iterations of the TDQM cycle focused on column integrity. The automated tool that Glocom embedded in the process was the Integrity Analyzer. Integrity violations were expected because the column integrity rules had not been consistently applied in the legacy systems. Table 4.1 summarizes the results from Glocom's many column integrity cycles in terms of the following five activities:

- *Identifying the problem.* The column integrity rule for cases booked, non-negative values, was tested using the Integrity Analyzer software to measure violations, producing the sample results in figure 4.2. Negative values were discovered in the CASE_BK (cases booked) field.
- *Diagnosing the problem.* The analyst examining these violations took them to the data quality manager for interpretation. From experience, the data quality manager surmised a possible legitimate meaning for the negative values: cancellations. He confirmed this interpretation with sales managers.
- *Planning the solution.* Using this diagnosis, the analyst and managers searched for a solution that would capture cancellations but not interfere with the standard interpretation of cases booked. They decided to create a new field for cancellations and remove these data from the "cases booked" field.
- *Implementing the solution.* This change was reflected at all relevant data levels—the operational database as well as the data warehouse.
- *Reflecting and learning.* Using an explicit iterative process for column integrity improvement facilitated communications between the data quality manager and the sales managers, which facilitated further column integrity improvement cycles. The data quality manager also learned to use the process-embedded data integrity tool to add new constraints and modify constraints as new requirements developed. For example, to detect new values created by sales, he programmed additional

Table 4.1

A Process-Embedded Column Integrity Example

Identifying the problem	For shipment data, CASE_BK column, acceptable values should not be negative. (<i>Define</i>) Check shipment data, CASE_BK column, for values that are less than zero. (<i>Measure</i>)
Diagnosing the problem	Review of violation records shows negative values for number of cases booked. (<i>Analyze</i>) Data quality manager communicates with sales managers to determine the validity of negative bookings. (<i>Analyze</i>)
Planning the solution	Sales managers confirm that negative values represent canceled bookings. (<i>Analyze</i>) Data quality manager and sales managers jointly decide, “This is confusing.” (<i>Analyze</i>) Data quality manager with approval from sales managers decides to add new field for canceled bookings. (<i>Analyze</i>)
Implementing the solution	New field for canceled bookings, CASE_CNL, is added to the shipment data. (<i>Improve</i>) Canceled bookings are removed from CASE_BK column and added to the new field. (<i>Improve</i>)
Reflecting and learning	Data quality manager learned to use the process-embedded data integrity tool to support problem solving. Sales managers learned to understand processes for exceptional cases and to communicate explicitly with data quality manager. Use of explicit iterative process for column integrity improvement facilitated communications between data quality manager and sales managers.

	CASE_BK	DOL_BK	AST_CUST_ID	PROD_ID	WK_ID
▶	-17	0	69000	90095	199841
	-7	0	69000	90096	199841
	-6	0	155450	1943	199738
	-1	0	348000	90097	199643
	-12	0	465600	90095	199633
	-49	0	541800	90095	199710

Figure 4.2

Sample Column Integrity Results

user-defined rules into the tool. Over time the sales managers learned the processes for exceptional cases and communicated directly with the data quality manager.

This example illustrates a common problem in legacy data over time: fields are used for purposes other than originally intended. The dynamic nature of data drives this behavior. In this example column integrity for the cases booked remained as initially defined, and a new field was added to represent canceled bookings. In other cases the column integrity definition may be adjusted to fit the revised field use. With automatically enforced integrity in relational databases, if a salesperson needs to enter data not within range, such as negative bookings, he should follow a standard procedure of requesting a database change. Organizations should be aware, however, that integrity constraints can be misused in relational database management systems, for example, by recording “cancellation” in a comment field and using a positive value. Thus, regular monitoring of column integrity is still necessary.

Concluding Remarks

In this chapter we suggested some canonical forms that quantitative metrics can take on, and presented instantiations of these forms for a sample group of data quality dimensions.

Having well-defined metrics, however, does not eliminate the need to perform the actual measurements. One would prefer to have an automated system to measure these dimensions. To make the necessary measurements manually on a relatively large database can be an onerous task. To illustrate one approach to automating the process of large-scale measurement, we presented a prototype package, the Integrity Analyzer, which has been used by a number of organizations in their assessment of data quality. We further illustrated the technique of embedding data integrity software, specifically the Integrity Analyzer, in the TDQM process.

If one is dealing with a large-scale measurement task with a very large database, it may not be feasible to examine every unit of data, and accepted sampling techniques must be applied. Chapter 5 discusses auditing the data.